arXiv:2510.12336v1 [quant-ph] 14 Oct 2025

Implementing the Quantum Approximate Optimization Algorithms for QUBO problems Across Quantum Hardware Platforms: Performance Analysis, Challenges, and Strategies

TEEMU PIHKAKOSKI, Nano and Molecular Systems Research Unit, University of Oulu, Finland
ARAVIND PLATHANAM BABU, Nano and Molecular Systems Research Unit, University of Oulu, Finland
Department of Instrumentation and Applied Physics, Indian Institute of Science, Bangalore, India
PAULI TAIPALE, OP Financial Group, Finland
PETRI LIIMATTA, OP Financial Group, Finland
MATTI SILVERI, Nano and Molecular Systems Research Unit, University of Oulu, Finland

Quantum computers are expected to offer significant advantages in solving complex optimization problems that are challenging for classical computers. Quadratic Unconstrained Binary Optimization (QUBO) problems represent an important class of problems with relevance in finance and logistics. The Quantum Approximate Optimization Algorithm (QAOA) is a prominent candidate for solving QUBO problems on near-term quantum devices. In this paper, we investigate the performance of both the standard QAOA and the adaptive derivative assembled problem tailored QAOA (ADAPT-QAOA) to solve QUBO problems of varying sizes and hardnesses with a focus on its practical applications in financial feature selection problems. Our main observation is that ADAPT-QAOA significantly outperforms QAOA with hard problems (trade-off parameter $\alpha = 0.6$) when comparing approximation ratio and time-to-solution. However, the standard QAOA remains efficient for simpler problems. Additionally, we investigate the practical feasibility and limitations of QAOA by scaling analysis based on the real-device calibration data for various hardware platforms. Our estimates indicate that standard QAOA implemented on superconducting quantum computers provides a shorter time-to-solution compared to trapped-ion devices. However, trapped-ion devices are expected to yield more favorable error rates. Our findings provide a comprehensive overview of the challenges, trade-offs, and strategies for deploying QAOA-based methods on near-term quantum hardware.

Additional Key Words and Phrases: QAOA, ADAPT-QAOA, quantum algorithms, QUBO problems, feature selection, performance analysis

1 Introduction

Optimization problems exist in almost every field, including finance, logistics, computational biology, and materials science. For example, in finance, a feature selection problem can model selecting the most relevant variables from a large set of candidate variables to assess the risk of a loan request. A bank may consider hundreds of potential indicators for default risk, including factors such as income levels, debt-to-income ratios, and payment history, among others. However, not all variables hold equal significance, and the objective of optimization is to select a subset of features that maximizes prediction accuracy and facilitates correct decision making. Many such optimization problems can be formulated as Quadratic Unconstrained Binary Optimization (QUBO) form, represented by a matrix encoding linear terms for each variable and quadratic terms representing correlations between variables [1–3]. QUBO problems belong to the class of NP-hard problems [1]. Widely known NP-hard problems, such as the Max-Cut problem or the Travelling Salesperson Problem (TSP), can be generalized as QUBOs. There is no known polynomial-time algorithm for solving

Authors' Contact Information: Teemu Pihkakoski, teemu.pihkakoski@oulu.fi, Nano and Molecular Systems Research Unit, University of Oulu, Oulu, Finland; Aravind Plathanam Babu, aravind.babu@oulu.fi, Nano and Molecular Systems Research Unit, University of Oulu, Oulu, Finland and Department of Instrumentation and Applied Physics, Indian Institute of Science, Bangalore, India; Pauli Taipale, OP Financial Group, Helsinki, Finland; Petri Liimatta, OP Financial Group, Helsinki, Finland; Matti Silveri, Nano and Molecular Systems Research Unit, University of Oulu, Oulu, Finland.

general QUBO problems deterministically. Classical numerical solvers can still find approximate solutions, but they fail or encounter convergence difficulties when the problem size increases, requiring more time and memory resources to obtain reasonable solutions.

Various quantum algorithms and algorithms have demonstrated the potential to offer a promising alternative to classical heuristics for solving QUBO problems. For example, Ref. [4] compared compared the performance of quantum annealing methods versus classical solvers by solving binary linear programming problems with quadratic constraints, which were converted into QUBO problems, and the quadratic constraints were handled by adding appropriate penalties to the QUBO formulation. They used quantum annealing methods, more specifically D-Wave's hybrid solver, and showed that a binary linear programming problem with approximately 600 variables and a single quadratic constraint can be solved in a shorter time using the D-Wave hybrid solver than with classical methods such as CPLEX, but at the expense of some loss in solution quality. Similarly, gate-based approaches like the Quantum Approximate Optimization Algorithm (QAOA), a hybrid quantum-classical variational algorithm [5] is a promising alternative. This algorithm uses parameterized quantum circuits to explore the solution space and to find high-quality approximate solutions. Many previous works have implemented methods to study the performance of QAOA in solving optimization problems on quantum simulators or real quantum computers [6–8]. However, research into finding optimization problems in which QAOA could outperform classical solvers is still ongoing. For example, QAOA has been shown to scale better than branch-and-bound solvers in terms of runtime when solving Low Autocorrelation Binary Sequences (LABS) problems [9]. Many variants of the QAOA aim to improve the performance and scalability of the algorithm [10]. Adaptive QAOA (ADAPT-QAOA) is one of these variants, and it enhances the efficiency of finding the solution by dynamically selecting quantum gates, reducing the number of variational parameters required [11].

We still face a considerable gap between the performance of current quantum hardware and algorithms in general. To close this gap and to improve the algorithms, we need more analysis on behavior of algorithms in various quantum use-cases and quantum computer implementations. Furthermore, from the business interest, now is an excellent time for resource estimation, as well as algorithm and hardware selection studies for potential practical use-cases. To this end, in this work, we compare the performance of the standard QAOA and ADAPT-QAOA in solving QUBO problems, concentrating on implementation. Our focus is on a feature selection problem inspired in its relevance in finance applications. To compare the performance of the algorithms, we use two metrics, the approximation ratio and the time-to-solution. The approximation ratio is used to assess the accuracy of the solution that the algorithm outputs, while the time-to-solution tells about the computational resources needed to run the algorithm. We examine how the performance of these algorithms varies with the size and hardness of the problem using a quantum simulator. We also incorporate calibration data and topologies from real quantum computers to further evaluate the theoretical performance and scaling of standard QAOA implementations in practical scenarios, providing insight into their potential application for real-world optimization problems now and in the near-future. We evaluate the theoretical performance of these devices by estimating the time-to-solution and total error probability when solving the QUBO problems.

The remaining content of the paper is organized as follows. First, in Section 2, we introduce the QUBO problems and their mathematical formulation. We also specify how to formulate feature selection problems as QUBO problems. In Section 3, we discuss the implementations of standard QAOA and ADAPT-QAOA to solve feature selection problems. In Section 4, we present experiments and results of the performance of standard QAOA and ADAPT-QAOA when solving feature selection problems, and we also estimate the theoretical performance of real quantum computers when using standard QAOA. Finally, in Section 5, we close with conclusions and outlook.

2 QUBO and feature selection problems

In general, Quadratic Unconstrained Binary Optimization (QUBO) problems can be represented using an upper-triangular matrix Q and a decision vector $\bar{x}$ [12]. The elements of the vector $\bar{x}$ are binary variables and they encode the solution to the problem. A general QUBO problem is solved by minimizing the objective function

$$f_{\text{QUBO}}(\bar{x}) = \sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j, \tag{1}$$

where Q_i and Q_{ij} are the diagonal and off-diagonal elements of the matrix Q. By definition QUBO forms are unconstrained. However, if needed, constraints can be handled by encoding them as additional penalty terms within the objective function [1].

In this work, we mainly consider feature selection problems [13]. In feature selection problems, we have a matrix in which the diagonal elements represent the cost of selecting a feature, and the off-diagonal elements represent the cost of selecting the corresponding two features, which either reward or penalize selecting the features together. The goal of the problem is to select a subset of features from all features so that the cost function f_{FS} is minimized. Feature selection problems are solved by minimizing the objective function

$$f_{\text{FS}}(\bar{x}) = -(1-\alpha) \sum_i Q_{ii} x_i + \alpha \sum_{i<j} Q_{ij} x_i x_j, \tag{2}$$

where $\alpha \in [0, 1]$ is a real-valued trade-off parameter. When α is closer to 0, the problem focuses more on finding relevant individual features, which are defined by the linear terms in the problem. In this case, problems with relatively low number of features are generally easy to solve. When α is closer to 1, the problem rewards more on finding solutions that minimize redundancy between variables, that is, the focus is on minimizing the sum of quadratic terms in the problem. Consequently, in general, the problem becomes more combinatorial, and thus harder to solve. In this work, we examine the simplest form of feature selection problems, that is, without additional constraints. For our experiments, we consider randomly generated Q matrices for the feature selection problems. For every problem instance, we use the same Q matrices throughout, varying only the trade-off parameter α.

3 Implementing QAOA to solve feature selection problems

To solve QUBO problems using the Quantum Approximate Optimization Algorithm (QAOA), the variables of the problem are converted from binary values to eigenvalues of the $\hat{\sigma}_z$ operator (± 1), that is, the variables of the problem are mapped to be represented by the states of the qubits. This can be done by converting the QUBO objective function into a mathematically equivalent Ising Hamiltonian using a transformation of variables

$$x_i \rightarrow \frac{1 - \hat{Z}_i}{2}, \tag{3}$$

where $\hat{Z}_i$ is the Pauli-Z gate acting on a qubit i [14]. Performing this transformation to Eq. (2), we obtain the Ising Hamiltonian $\hat{H}_C$ which can be used as the cost Hamiltonian to solve the feature selection problem using QAOA,

$$\hat{H}_C = -(1-\alpha) \sum_i Q_{ii} \frac{1 - \hat{Z}_i}{2} + \alpha \sum_{i<j} Q_{ij} \frac{1 - \hat{Z}_i}{2} \frac{1 - \hat{Z}_j}{2}. \tag{4}$$

Next, we cover the steps needed to implement QAOA and ADAPT-QAOA to solve optimization problems.

3.1 Standard QAOA

The standard QAOA is constructed from initialization, cost and mixer layers, and optimization [5, 10]. A single QAOA layer consists of a cost layer followed by a mixer layer. Formally, in a QAOA layer k, the cost layer $\hat{U}_C(\gamma_k)$ and the mixer layer $\hat{U}_M(\beta_k)$ are defined as

$$\hat{U}_C(\gamma_k) = e^{-i\gamma_k \hat{H}_C}, \qquad \hat{U}_M(\beta_k) = e^{-i\beta_k \hat{H}_M}, \tag{5}$$

where γ_k and β_k are variational parameters. Here $\hat{H}_M$ is the mixer Hamiltonian, which in the standard QAOA is always a rotational Pauli-X gate acting on every qubit in the circuit,

$$\hat{H}_M = \sum_{i=1}^{n} \hat{X}_i, \tag{6}$$

where n is the total number of qubits that is equal to the total number of features in the feature selection problem. We construct the standard QAOA in an iterative way, which means that we append a QAOA layer one at a time and optimizing all variational parameters after each addition, similar to what is done in ADAPT-QAOA [11]. To implement the first layer of the standard QAOA, we start by initializing every qubit in the circuit to an equal superposition by applying a Hadamard gates (H) as shown in Fig. 1:

$$|\psi_0\rangle = \left(\hat{H}\,|0\rangle\right)^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)^{\otimes n}. \tag{7}$$

After this, we apply the first QAOA layer, and the state of the system reads

$$|\psi_1\rangle = e^{-i\beta_1 \hat{H}_M} e^{-i\gamma_1 \hat{H}_C}\,|\psi_0\rangle\,, \tag{8}$$

where γ_1 is initially a random value in the range $[0, 2\pi]$ and β_1 is initially a random value in the range $[0, \pi]$. The variational parameters are updated using a classical optimization algorithm with the goal of minimizing the expectation value of the cost Hamiltonian $\hat{H}_C$ with respect to the state of the circuit. The expectation value of the cost Hamiltonian with respect to the variational state $|\psi_k\rangle$ in layer k is defined as

$$C_k = \left\langle \psi_k \left| \hat{H}_C \right| \psi_k \right\rangle. \tag{9}$$

The optimized variational parameters of layer k serve as the initial values for the variational parameters for the next layer $k+1$ with the addition of parameters γ_{k+1} and β_{k+1} initialized to random values in the ranges $[0, 2\pi]$ and $[0, \pi]$.

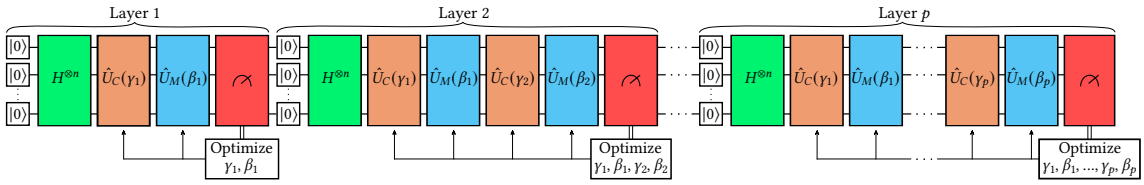

Fig. 1. The standard QAOA can be divided into three parts, initialization (colored green), the application of QAOA layers (colored orange and blue), and optimization (colored red). In the initialization step, every qubit is initialized to an equal superposition state by applying a Hadamard gates (H). After the initialization, cost and mixer layers are applied to the circuit. In the optimization step, a classical optimization method is used to update the variational parameters so that the expectation value of the cost Hamiltonian with respect to the variational state is minimized. In this work, we apply the layers iteratively, optimizing all variational parameters after each addition.

The algorithm ends when all the variational parameters for the last layer p have been optimized. Physically, the principle idea of QAOA is to generate a time evolution $\hat{U}_{\mathrm{C}}(\gamma_p)\hat{U}_{\mathrm{M}}(\beta_p)\ldots\hat{U}_{\mathrm{C}}(\gamma_1)\hat{U}_{\mathrm{M}}(\beta_1)$ such that it steers the ground state of the mixer Hamiltonian $\hat{H}_{\mathrm{M}}$ to the ground state of the cost Hamiltonian $\hat{H}_{\mathrm{C}}$.

The solution quality of QAOA is evaluated using the approximation ratio r_k which is given by

$$r_k = \frac{C_k}{C_{\text{exact}}}, \tag{10}$$

where C_k is the expectation value obtained at the end of layer k, and C_{exact} is the exact solution value, obtained in this work using a classical solver software Gurobi OptiMods [15]. For the problem sizes that we consider in this work, the classical approach solves the problems exactly sufficiently fast.

3.2 ADAPT-QAOA

The adaptive derivative assembled problem tailored QAOA (ADAPT-QAOA) is a variant of the standard QAOA, in which the mixer Hamiltonian is adaptively chosen for each QAOA layer [10, 11]. This algorithm has been shown to converge to better solutions in MaxCut problems with regular graphs than standard QAOA in the same number of layers [11]. The mixer for each QAOA layer is selected from a mixer pool which consists of

- The standard QAOA X-mixer along with a variant that uses Y-gates, which are applied to all qubits in the circuit, $P_{\mathrm{XY}} = \left\{\sum_i^n \hat{X}_i\right\} \cup \left\{\sum_i^n \hat{Y}_i\right\}$,
- Single-qubit mixers, $P_{\text{single}} = \bigcup_i \left\{\hat{X}_i, \hat{Y}_i\right\}$,
- Two-qubit mixers, $P_{\text{two}} = \bigcup_{i\neq j} \left\{B_i C_j \mid B, C \in \left\{\hat{X}, \hat{Y}, \hat{Z}\right\}\right\}$.

The complete mixer pool for ADAPT-QAOA is $P_{\text{pool}} = P_{\text{two}} \cup P_{\text{single}} \cup P_{\mathrm{XY}}$. In P_{XY}, there are two mixers, independent of the number of qubits n. In P_{single}, there are $2n$ mixers, which is straightforward to see. For the two-qubit mixers, there are 3 choices for the operator on qubit i and 3 choices for the operator on qubit j, which amounts to 9 distinct two-qubit operators on the pair $\{i, j\}$. The number of qubit pairs is $\binom{n}{2} = \frac{n(n-1)}{2}$, which amounts to $9\frac{n(n-1)}{2} = 9\frac{n^2-n}{2}$ mixers in P_{two}. Overall, the number of mixers in the mixer pool is $2 + 2n + 9\frac{n^2-n}{2}$ scaling as $O(n^2)$.

The mixer is selected by using a maximum gradient criterion. For QAOA layer k, the energy gradient for the lth mixer $\hat{A}_l$ in the mixer pool is given by

$$g_l = \left|-i\left\langle\psi_{(k-1)}\left|e^{i\hat{H}_C\gamma_0}\left[\hat{H}_C, \hat{A}_l\right]e^{-i\hat{H}_C\gamma_0}\right|\psi_{(k-1)}\right\rangle\right|, \tag{11}$$

where $|\psi_{(k-1)}\rangle$ is the state of the system at the end of layer $k-1$, and the parameter γ_0 is set to a small value close to zero, such as $\gamma_0 = 0.01$. The mixer giving the maximum gradient is selected as the mixer Hamiltonian for the layer k. It is believed that the mixer selection reduces excitations to higher-excited states during the algorithm improving its accuracy and speed [11].

4 Experiments

In all experiments, we used standard QAOA or ADAPT-QAOA up to 30 layers. We used the ideal quantum simulator of QisKit [16] with 10^4 shots per optimization iteration. For the classical optimization of the variational parameters γ_k and β_k, we used the Powell method from the SciPy library [17] and limited the number of iterations per layer to a maximum of 1500. By decreasing the number of shots per optimization iteration or the maximum number of optimization iterations per layer, the algorithms can be sped up with the cost of potentially decreasing the accuracy of the algorithms. However, this does not change the scaling of time-to-solution as a function of the problem size, which is what we are interested

in. For each size of the feature selection problem, we generated data for the QUBO matrices using 10 different seeds for a random number generator, always using the same set of seeds in all experiments.

4.1 Standard QAOA vs ADAPT-QAOA in solving feature selection problems

We used approximation ratio and time-to-solution as the metrics to compare the performance of standard QAOA and ADAPT-QAOA in solving feature selection problems. We considered problems with 6, 10 and 14 features, and with $\alpha = 0.2$ and $\alpha = 0.6$, and the results can be seen in Fig. 2. With $\alpha = 0.2$, there is not much difference in the approximation ratios between the two algorithms. However, as the hardness of the problem increases with $\alpha = 0.6$, a notable difference arises in the approximation ratios obtained by the algorithms. This suggests that the ability to pick a mixer more freely from a mixer pool provides ADAPT-QAOA the ability to converge to a better solution in the same amount of layers as in standard QAOA. In each of the problem sizes, at least one problem instance in which ADAPT-QAOA reached an approximation ratio very close to 1, and the maximum approximation ratio of ADAPT-QAOA was always higher than that of standard QAOA.

In general, ADAPT-QAOA with 30 layers achieves higher-quality solutions on average than standard QAOA for these feature selection problems. However, how do the two algorithms compare in terms of time-to-solution? With 6 features, the time-to-solution of standard QAOA and ADAPT-QAOA are quite close to each other, but as the problem size increases, ADAPT-QAOA starts to take a longer time to solve the problem compared to standard QAOA. This arises again from the additional mixer selection step in ADAPT-QAOA, where the size of the mixer pool increases with the problem size. As a result, larger problems lead to more possible mixers to choose from in each layer, making the mixer selection process take more time. Similar kind of increase in time-to-solution is also shown in the case of 1 layer of standard QAOA or ADAPT-QAOA as the problem size increases.

These results suggest that it is preferable to use ADAPT-QAOA over standard QAOA when solution quality is prioritized over time-to-solution. This advantage becomes more pronounced as α increases, demonstrating that ADAPT-QAOA outperforms standard QAOA more significantly in terms of solution quality as the hardness of the problem increases.

To better highlight the performance differences between standard QAOA and ADAPT-QAOA, in Fig. 3 we show the approximation ratio and the time-to-solution when solving feature selection problems with 14 features with standard QAOA or ADAPT-QAOA with layers ranging from 1 to 30. For example, with $\alpha = 0.6$, it can be seen that 15 layers of ADAPT-QAOA performs better than 30 layers of standard QAOA in both approximation ratio and time-to-solution. In Fig. 4, we show the results for solving feature selection problems with 6, 10 and 14 features with $\alpha = 0.2$ and $\alpha = 0.6$ using 15 layers of standard QAOA and 30 layers of ADAPT-QAOA. With $\alpha = 0.6$, ADAPT-QAOA consistently outperforms standard QAOA in terms of both approximation ratio and time-to-solution across all problem sizes considered, highlighting that as the problem hardness increases, ADAPT-QAOA with fewer layers than used in standard QAOA can achieve better approximation ratio and time-to-solution.

4.2 Classical QUBO solver

We also solved the feature selection problems using a classical QUBO solver Gurobi OptiMods [15]. This solver uses heuristic methods to provide initial solutions quickly and then uses branch-and-bound methods to explore the solution space to tighten the bounds on the optimal solution. The algorithm ends once an optimality gap is within a predefined

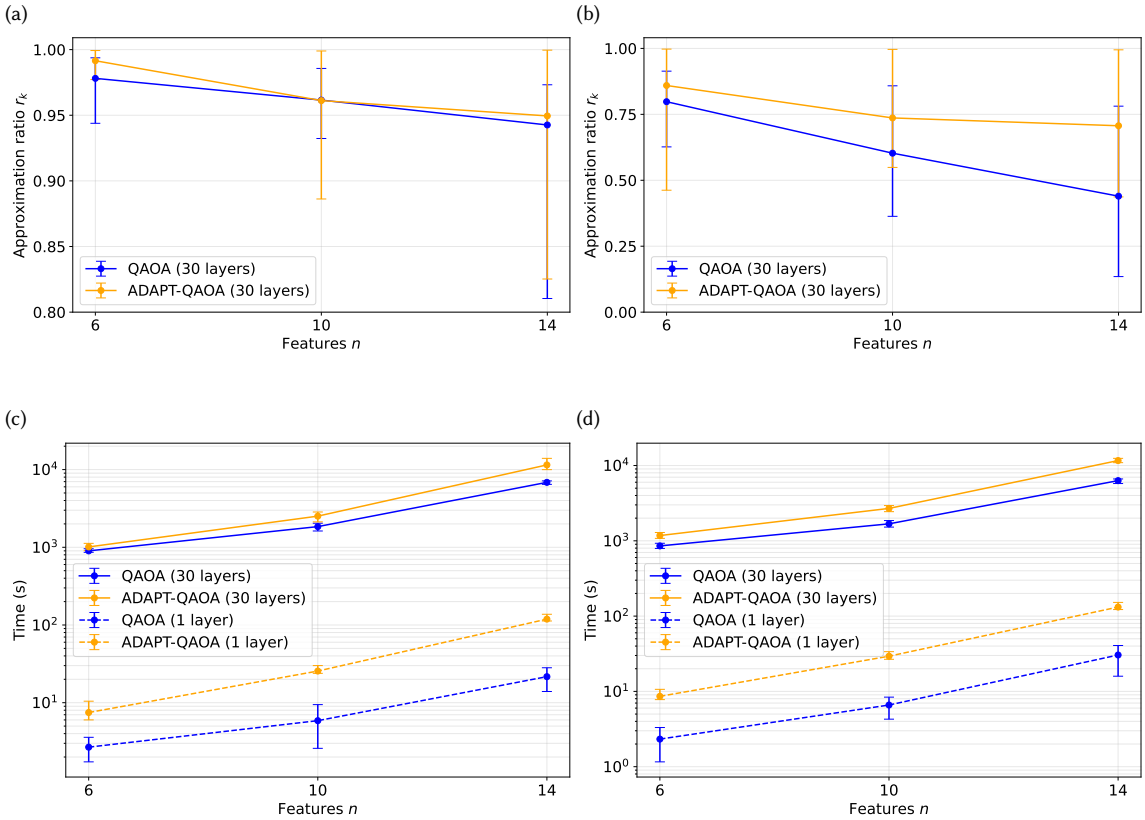

Fig. 2. Mean approximation ratio r_k (a-b) and time-to-solution (c-d) with respect to the number of features n using 30 layers of standard QAOA and ADAPT-QAOA to solve feature selection problems with trade-off parameter set to $\alpha = 0.2$ (a), (c) and 0.6 (b), (d). Time-to-solution is given also for 1 layer of standard QAOA and ADAPT-QAOA, for comparison. All results are averaged over 10 problem instances obtained using 10 different seeds to generate data for the problems. The vertical bars represent the minimum and maximum values obtained in each case.

tolerance, where the gap is defined as

$$\text{gap} = \frac{|z_{\text{bound}} - z_{\text{best}}|}{|z_{\text{best}}|}, \tag{12}$$

where z_{best} is the objective value of the best solution found and z_{bound} is the bound on the optimal objective value. When solving the problems with this solver, we used 10^{-4} as the optimality gap. Using this classical solver, we found that it had considerably better scaling in terms of time-to-solution than the standard QAOA or ADAPT-QAOA when solving the feature selection problems as we have formulated them. This provides motivation for the future to reformulate the problem in ways that make it more challenging for classical solvers, yet still feasible to solve using the quantum algorithms.

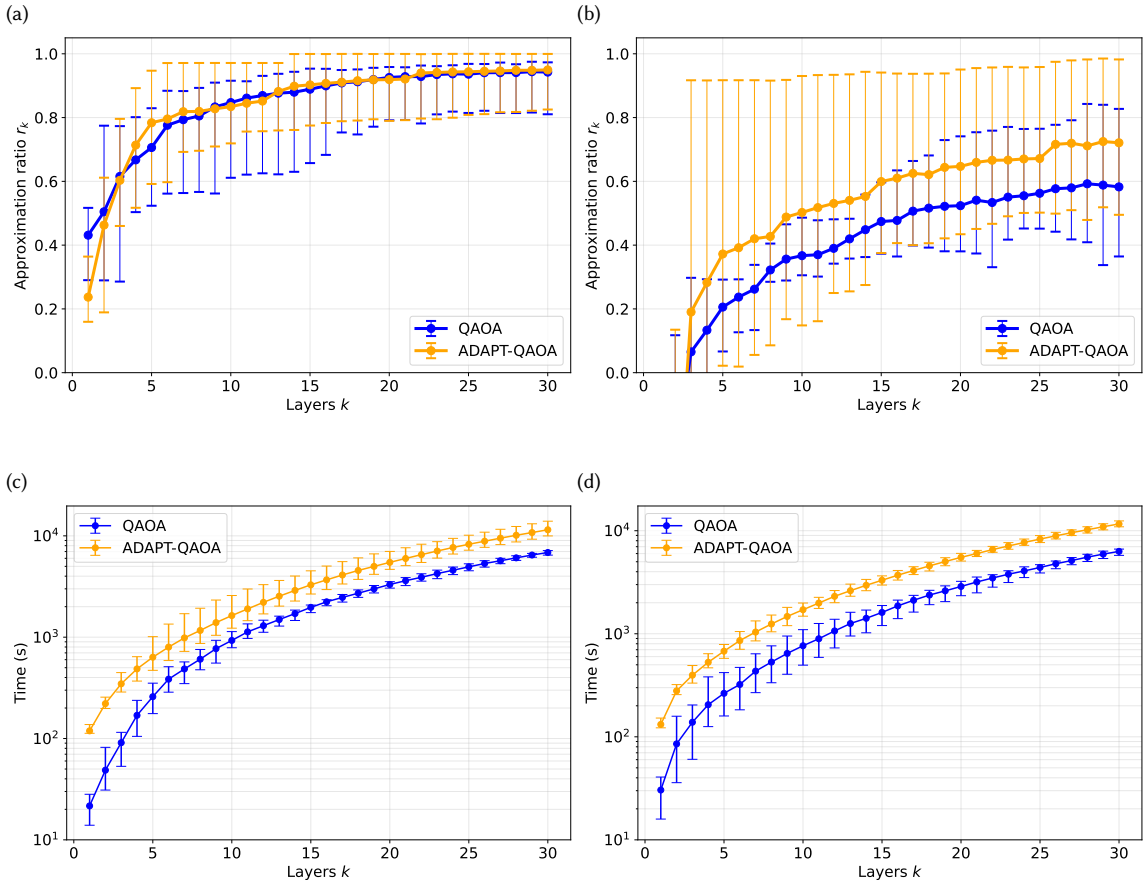

Fig. 3. Mean approximation ratio r_k (a-b) and time-to-solution (c-d) with respect to the number of layers k when solving feature selection problems with 14 features using standard QAOA and ADAPT-QAOA with trade-off parameter set to $\alpha = 0.2$ (a), (c) and 0.6 (b), (d). All results are averaged over 10 problem instances obtained using 10 different seeds to generate data for the problems. The caps connected with dotted lines represent the minimum and maximum values obtained in each case.

4.3 Comparison of different topologies and architectures

In order to understand how the topology and the gate durations of different quantum computers affect the performance of solving QUBO problems with QAOA, we also estimated the theoretical performance of the implementations of QAOA and ADAPT-QAOA algorithms on multiple quantum computers. The quantum computers, for which we calculated the estimates, were the Quantinuum H2 trapped-ion device [18] and the ibm_brisbane superconducting device [19], which have all-to-all and heavy-hex topologies, respectively. Generally, gate operations with trapped-ion qubits are slower than with superconducting qubits, due to the additional time needed to transport qubits in trapped-ion architectures [20]. In addition, we considered theoretical implementations of the ibm_brisbane device with square lattice and all-to-all topologies, to see how the qubit topology changes the performance of the device. Of these topologies, the heavy-hex topology has the least connections between qubits (on average 2.3), the square lattice topology has more connections (4)

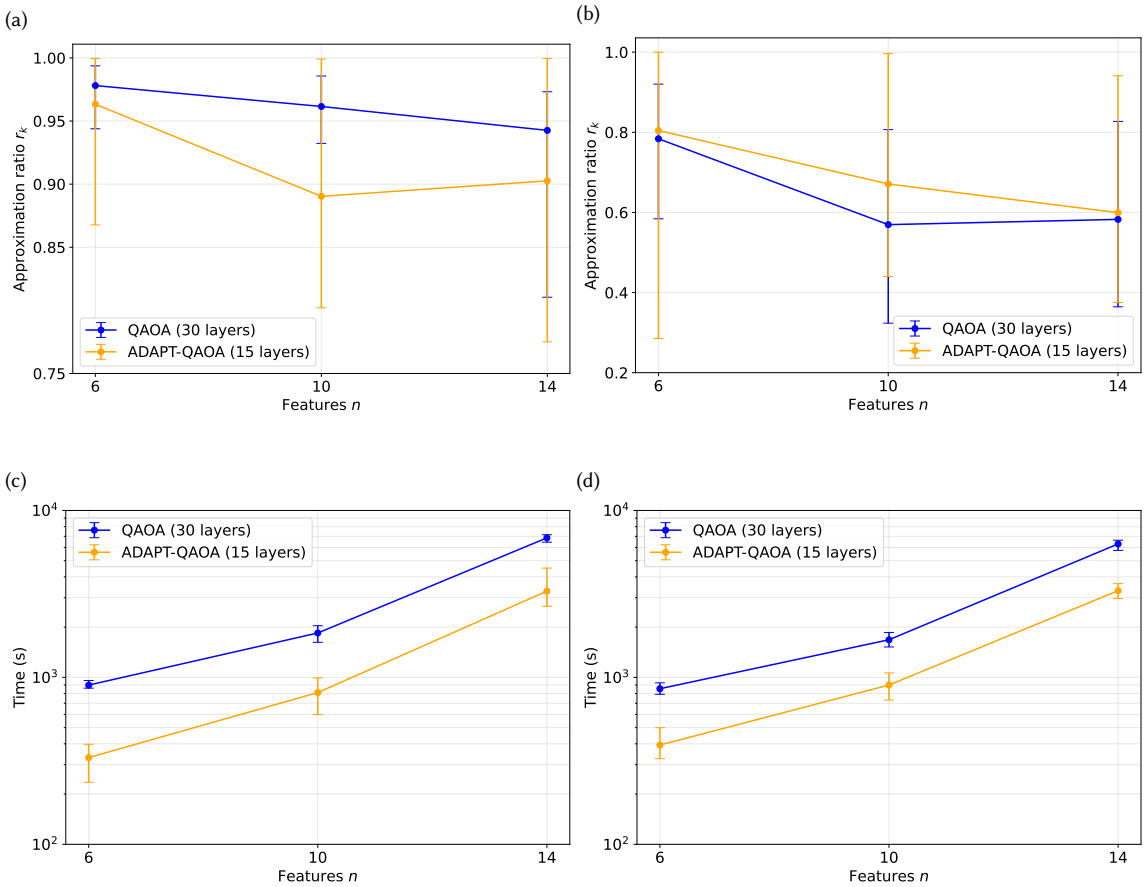

Fig. 4. Mean approximation ratio r_k (a-b) and time-to-solution (c-d) with respect to the number of features n using 30 layers of standard QAOA and 15 layers of ADAPT-QAOA to solve feature selection problems with trade-off parameter set to $\alpha = 0.2$ (a), (c) and 0.6 (b), (d). All results are averaged over 10 problem instances obtained using 10 different seeds to generate data for the problems. The vertical bars represent the minimum and maximum values obtained in each case.

than the heavy-hex topology, and lastly, in the all-to-all topology, every qubit is connected to all other qubits. Visual representations of the heavy-hex and square lattice topologies are shown in Fig. 5. Due to the limited amount of connections in heavy-hex and square lattice topologies, additional SWAP gates are needed to implement two-qubit gates if the qubits on which the gate operates are not connected. This generally results in a larger amount of operations needed to construct an algorithm, which in turn results in a longer time-to-solution.

During the optimization part of a layer, N optimization iterations are performed. In each of the optimization iterations, S shots of the quantum circuit are measured. The time to execute a layer on a quantum computer can be estimated by

$$T_{\text{layer}} = (d_1 t_1 + d_2 t_2) N S, \tag{13}$$

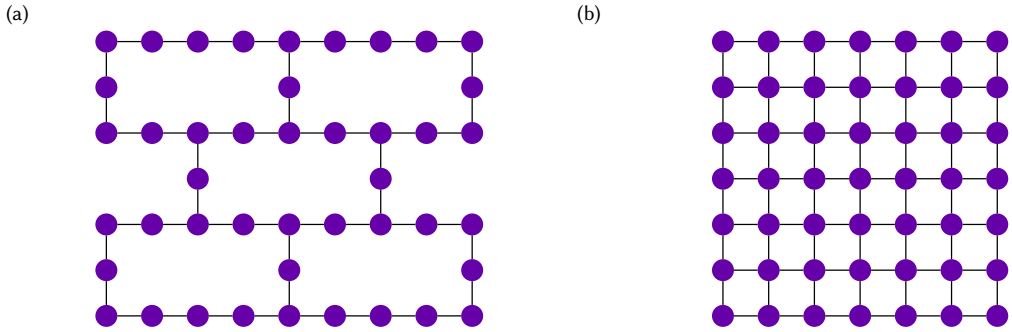

Fig. 5. Visualization of (a) heavy-hex and (b) square lattice topologies. Purple nodes represent qubits.

where d_1 and d_2 are the number of single-qubit and two-qubit gate layers in the QAOA circuit and t_1 and t_2 are the durations of single-qubit and two-qubit gates for the quantum computer.

The number of gate layers are extracted from the quantum circuits used in the simulation of standard QAOA in solving feature selection problems by grouping gates into layers such that, within each layer, at most one gate acts on any given qubit. This ensures that all gates in the same layer can be applied simultaneously. In this estimation, the additional time due to state preparation and measurement overheads is left out, which we justify by the fact that they are approximately constant and short compared to the total time of gate durations. The gate durations t_1 and t_2 for the ibm_brisbane quantum computer were obtained from the calibration data fetched from IBM Quantum [19], and similar data for the Quantinuum H2 quantum computer was obtained from Ref. [20], see Table 1. For the Quantinuum H2 quantum computer, the single-qubit gate duration was calculated by the sum of the intrazone shift time and the single-qubit gate time and the two-qubit gate duration was calculated by the sum of the interzone shift time and the two-qubit gate time. In the devices we consider in this work, the Z-gate can be implemented by changing the reference frame of the control electronics, thus changing the phase of all subsequent gates that act on that qubit [21, 22]. This means that the Z-gates practically do not cost any time and do not introduce any additional error, which is taken into account in these estimations. An estimate of the total time to execute the QAOA algorithm is determined by the total time required to execute all layers.

In Fig. 6, it can be seen that with low problem sizes, the Quantinuum H2 quantum computer is estimated to take the longest amount of time to solve the problems using 30 layers of standard QAOA. The single-qubit and two-qubit gate durations of the device, which are on the order of microseconds, are 1 to 2 magnitudes larger than the single-qubit

Table 1. Calibration data for ibm_brisbane and Quantinuum H2 quantum computers. Data for ibm_brisbane is obtained from IBM Quantum services [19] from a snapshot on 26 August 2025 and data for Quantinuum H2 is obtained from Refs. [18, 20].

Value		ibm_brisbane	Quantinuum H2
Single-qubit gate duration	(t_1)	0.06 μs	63 μs
Two-qubit gate duration	(t_2)	0.66 μs	308 μs
Single-qubit gate error	(e_1)	3.3×10^{-4}	3.0×10^{-5}
Two-qubit gate error	(e_2)	1.2×10^{-2}	1.0×10^{-3}
Measurement error	(e_m)	3.7×10^{-2}	1.0×10^{-3}

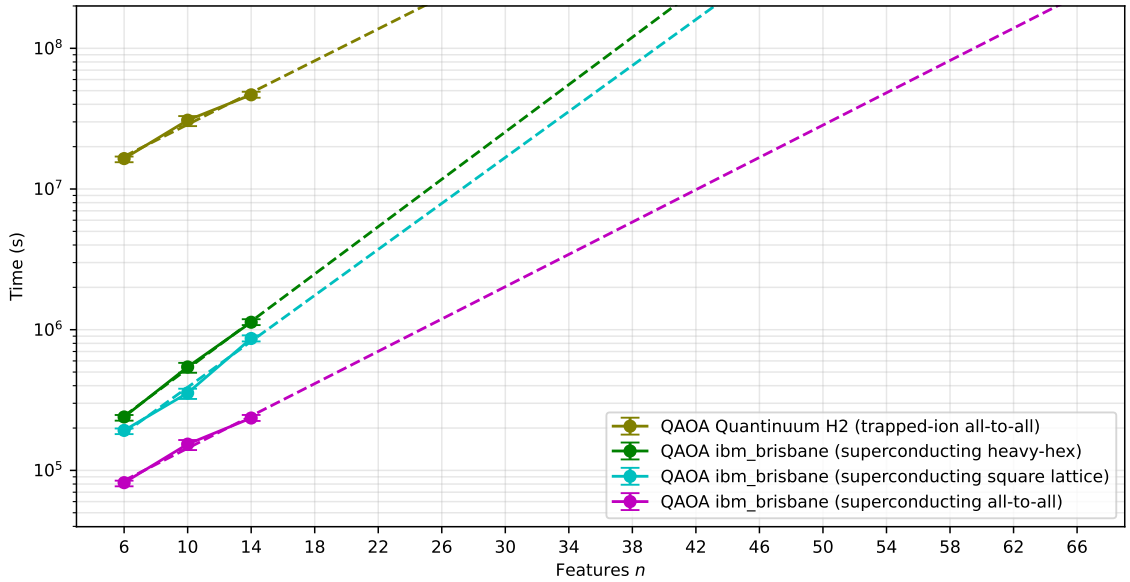

Fig. 6. Mean estimated time-to-solution for feature selection problems with $\alpha = 0.2$ solved using 30 layers of standard QAOA as a function of features n for the ibm_brisbane superconducting quantum computer and the Quantinuum H2 trapped-ion quantum computer. The ibm_brisbane has a heavy-hex topology and the Quantinuum H2 has an all-to-all topology. In addition, we calculated the estimates for theoretical square lattice and all-to-all topologies for the ibm_brisbane device. The dashed lines represent extrapolations derived from the data points. All results are averaged over 10 problem instances obtained using 10 different seeds to generate data for the problems. The vertical bars represent the minimum and maximum values obtained in each case.

and two-qubit gate durations of the ibm_brisbane quantum computer, which are on the order of nanoseconds. This explains the difference in the estimated time-to-solution of the Quantinuum H2 and the ibm_brisbane with an all-to-all theoretical topology. At this stage, the Quantinuum H2 does not appear to be the most suitable quantum computer for solving these types of optimization problems in terms of time-to-solution, especially since the real device has only 56 qubits. Compared to the ibm_brisbane quantum computer with the heavy-hex topology, the Quantinuum H2 would need more qubits to be able to gain an advantage in time-to-solution over the ibm_brisbane heavy-hex device. When comparing the heavy-hex and the theoretical square lattice topologies on the ibm_brisbane device, the square lattice device provides better scaling in terms of time-to-solution. This is expected since the square lattice topology has more connections between the qubits and thus needs fewer SWAP gates to implement the algorithm. Finally, from the quantum devices, the ibm_brisbane quantum computer with the theoretical topology of all-to-all connectivity performs the best in terms of time-to-solution.

4.4 Error estimation

To complement the time estimations for solving the feature selection problems using different quantum computers, we additionally estimated the errors occurring during the standard QAOA, providing a metric to evaluate and compare the solution quality achieved by the quantum computers. The total error probability [23] when executing a quantum circuit

can be estimated by

$$E_{\text{tot}} = 1 - \left[(1-e_1)^{N_1}(1-e_2)^{N_2}(1-e_m)^{N_m}\right], \tag{14}$$

where e_1, e_2 and e_m denote the error probabilities of single-qubit gates, two-qubit gates, and measurements, respectively, and N_1, N_2 and N_M represent the corresponding counts of single-qubit gates, two-qubit gates, and measurements within the circuit.

The error probabilities of single-qubit gates, two-qubit gates, and measurements for the ibm_brisbane quantum computer were obtained from the calibration data fetched from IBM Quantum [19], and similar data for the Quantinuum H2 quantum computer was obtained from Ref. [18], see Table 1. Using Eq. (14), we estimated the total error probabilities for solving a feature selection problems with 6 features using 1 layer of standard QAOA. We use 1 layer of standard QAOA to estimate the errors, since we do not consider using error mitigation methods which could have a notable impact on the errors when using, for example, 30 layers as we did in the estimation of time-to-solution. The estimated total error probabilities are shown in Fig. 7. When considering the ibm_brisbane quantum computer, the topology matters in the error estimation as topologies with fewer connections between qubits usually require more SWAP gates, and the all-to-all topology provides the lowest estimated total error probability for the ibm_brisbane quantum computer, as expected. The Quantinuum H2 quantum computer has an all-to-all topology and its gate error rates and measurement error rates are lower than those of the ibm_brisbane quantum computer, and thus provides the lowest estimated total error probability for solving the feature selection problem using standard QAOA.

In these error estimations, we did not consider the effect of error mitigation methods which are widely used to improve the fidelity of quantum computations when using real quantum computers. These methods include, for example, zero-noise extrapolation [24] and probabilistic error cancellation [25], which have their own advantages in improving the fidelity of the computations with the cost of additional classical and quantum computation time. This means that the estimated total error probabilities shown in Fig. 7 should be considered as upper bounds for the errors. Finally,

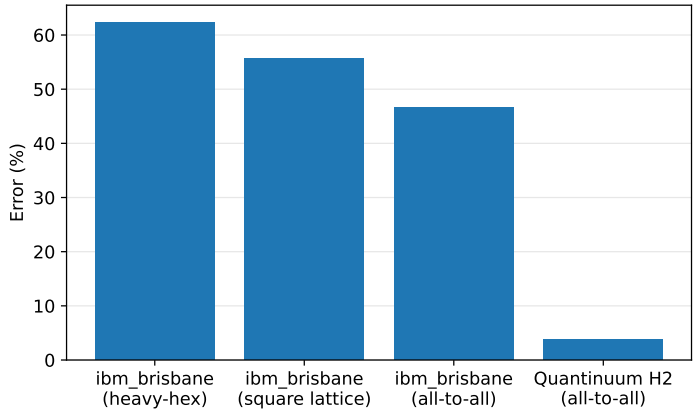

Fig. 7. Estimated total error probabilities for solving a feature selection problem with 6 features using 1 layer of standard QAOA on the ibm_brisbane with heavy-hex topology, hypothetical cases of ibm_brisbane with square lattice and all-to-all topology and the Quantinuum H2 quantum computers.

our results highlight the need for further improvements in gate error rates and measurement error rates in hardware development..

5 Conclusions

In this work, we investigated the performance of the standard QAOA and ADAPT-QAOA in solving QUBO problems, focusing on feature selection problems, and we estimated the performance of real quantum computers when solving the same problems using standard QAOA, taking into account the topologies, gate durations, and error rates of the devices.

Starting from the QUBO formulation of feature selection problems, we discussed how to encode feature selection problems into cost Hamiltonians which can be used to solve the problems using QAOA. We thoroughly explained the step-by-step implementation of standard QAOA, from initialization to optimization. We also described how to adaptively select the mixer Hamiltonian in ADAPT-QAOA using a gradient criterion.

Using an ideal simulator, we solved feature selection problem instances using standard QAOA and ADAPT-QAOA. We considered problem sizes of 6, 10 and 14 features, and trade-off parameters $\alpha = 0.2$ and $\alpha = 0.6$, as increasing the trade-off parameter makes the problem harder. Using different seeds for a random number generator, we generated 10 different problems for each problem size. With $\alpha = 0.2$, meaning that the problems are easier, 30 layers of standard QAOA and ADAPT-QAOA achieve quite similar approximation ratios on average. As the trade-off parameter is increased to $\alpha = 0.6$, ADAPT-QAOA achieves significantly better approximation ratios on average than standard QAOA. Notably, in each of the problem sizes and with both trade-off parameters, at least one instance of ADAPT-QAOA converged very close to the exact solution. With the same amount of layers, ADAPT-QAOA takes a longer time to run than the standard QAOA due to the additional mixer selection process. Furthermore, we compared the performance of 15 layers of ADAPT-QAOA and 30 layers of standard QAOA, and observed that as the problem hardness increases, ADAPT-QAOA outperforms standard QAOA in both average approximation ratio and time-to-solution. This suggests that standard QAOA is more suitable for easier problems, whereas ADAPT-QAOA becomes preferable as problem hardness increases where it can achieve better results than standard QAOA, using fewer layers.

Finally, we estimated the theoretical performance of the ibm_brisbane superconducting quantum computer and the Quantinuum H2 trapped-ion quantum computer in solving the feature selection problems using standard QAOA. We also considered theoretical square lattice and all-to-all topologies on the ibm_brisbane quantum computer. We estimated the time-to-solution and total error probabilities by extracting gate counts and circuit depth from the QAOA circuits, and by incorporating gate durations and error rates obtained from device calibration data. The gate operations of the Quantinuum H2 are relatively slow due to qubit shift operations, resulting in the longest time-to-solution in every problem size. However, the Quantinuum H2 has lower error rates than the ibm_brisbane, and thus achieves lower total error probabilities. We also found that the topology of the device matters, as increasing connectivity between the qubits in the ibm_brisbane results in lower time-to-solution and total error probability due to fewer SWAP operations in the quantum circuits. In the future, further improvements in qubit connectivity and gate and measurement error rates are essential to enhance the fidelity of quantum computations.

Comparing the performance of standard QAOA and ADAPT-QAOA to a classical solver Gurobi OptiMods, we found that the classical solver provides better scaling in terms of time-to-solution when solving the feature selection problems. To this end, more work is needed in finding problems that are harder to solve classically, but still tractable using quantum algorithms. An interesting direction would be to incorporate penalty terms into the QUBO objective function. Additionally, it is important to explore more efficient quantum algorithms, such as alternative variants of QAOA, and their susceptibility against various gate and qubit errors.

Acknowledgments

We acknowledge financial support through the projects Towards Reliable Quantum Software Development: Approaches and Use Case (TORQS), grant no. 8436/31/2022, and Securing the Quantum Software Stack (SeQuSoS), grant no. 102/31/2024, by Business Finland; H2Future, grant no. 352788, by the Research Council of Finland and the University of Oulu; and Classi|Q⟩, project number 24304728121, jointly by the City of Oulu and the University of Oulu. A. P. B. acknowledges the financial support from the Department of Science and Technology (DST), Government of India, through the INSPIRE Faculty Fellowship. We are grateful for useful discussion with Matti Raasakka and all members of the TORQS project team.

References

[1] Fred Glover, Gary Kochenberger, Rick Hennig, and Yu Du. Quantum bridge analytics I: a tutorial on formulating and using QUBO models. *Ann. Oper. Res.*, 314(1):141–183, July 2022. doi:10.1007/s10479-022-04634-2.

[2] Sascha Mücke, Raoul Heese, Sabine Müller, Moritz Wolter, and Nico Piatkowski. Feature selection on quantum computers. *Quantum Mach. Intell.*, 5(1):11, February 2023. doi:10.1007/s42484-023-00099-z.

[3] Erica Grant, Travis S. Humble, and Benjamin Stump. Benchmarking Quantum Annealing Controls with Portfolio Optimization. *Phys. Rev. Appl.*, 15(1):014012, January 2021. doi:10.1103/PhysRevApplied.15.014012.

[4] Finley Alexander Quinton, Per Arne Sevle Myhr, Mostafa Barani, Pedro Crespo del Granado, and Hongyu Zhang. Quantum annealing applications, challenges and limitations for optimisation problems compared to classical solvers. *Sci. Rep.*, 15(1):12733, April 2025. doi:10.1038/s41598-025-96220-2.

[5] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A Quantum Approximate Optimization Algorithm, November 2014. arXiv:1411.4028 [quant-ph]. doi:10.48550/arXiv.1411.4028.

[6] David Fitzek, Toheed Ghandriz, Leo Laine, Mats Granath, and Anton Frisk Kockum. Applying quantum approximate optimization to the heterogeneous vehicle routing problem. *Sci. Rep.*, 14(1):25415, October 2024. doi:10.1038/s41598-024-76967-w.

[7] Giuseppe Buonaiuto, Francesco Gargiulo, Giuseppe De Pietro, Massimo Esposito, and Marco Pota. Best practices for portfolio optimization by quantum computing, experimented on real quantum devices. *Sci. Rep.*, 13(1):19434, November 2023. doi:10.1038/s41598-023-45392-w.

[8] Jason Larkin, Matías Jonsson, Daniel Justice, and Gian Giacomo Guerreschi. Evaluation of QAOA based on the approximation ratio of individual samples. *Quantum Sci. Technol.*, 7(4):045014, August 2022. doi:10.1088/2058-9565/ac6973.

[9] Ruslan Shaydulin, Changhao Li, Shouvanik Chakrabarti, Matthew DeCross, Dylan Herman, Niraj Kumar, Jeffrey Larson, Danylo Lykov, Pierre Minssen, Yue Sun, Yuri Alexeev, Joan M. Dreiling, John P. Gaebler, Thomas M. Gatterman, Justin A. Gerber, Kevin Gilmore, Dan Gresh, Nathan Hewitt, Chandler V. Horst, Shaohan Hu, Jacob Johansen, Mitchell Matheny, Tanner Mengle, Michael Mills, Steven A. Moses, Brian Neyenhuis, Peter Siegfried, Romina Yalovetzky, and Marco Pistoia. Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem. *Sci. Adv.*, 10(22):eadm6761, May 2024. doi:10.1126/sciadv.adm6761.

[10] Kostas Blekos, Dean Brand, Andrea Ceschini, Chiao-Hui Chou, Rui-Hao Li, Komal Pandya, and Alessandro Summer. A review on Quantum Approximate Optimization Algorithm and its variants. *Phys. Rep.*, 1068:1–66, June 2024. doi:10.1016/j.physrep.2024.03.002.

[11] Linghua Zhu, Ho Lun Tang, George S. Barron, F. A. Calderon-Vargas, Nicholas J. Mayhall, Edwin Barnes, and Sophia E. Economou. Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer. *Phys. Rev. Res.*, 4(3):033029, July 2022. doi:10.1103/PhysRevResearch.4.033029.

[12] Mark Lewis and Fred Glover. Quadratic unconstrained binary optimization problem preprocessing: Theory and empirical analysis. *Netw.*, 70(2):79–97, September 2017. doi:10.1002/net.21751.

[13] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *J. Mach. Learn. Res.*, 3:1157–1182, March 2003.

[14] Andrew Lucas. Ising formulations of many NP problems. *Front. Phys.*, 2, February 2014. doi:10.3389/fphy.2014.00005.

[15] Gurobi Optimization LLC. Gurobi Optimizer Reference Manual, 2025. URL: https://www.gurobi.com/.

[16] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with Qiskit, 2024. arXiv:2405.08810 [quant-ph]. doi:10.48550/arXiv.2405.08810.

[17] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nat. Methods*, 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.

[18] Quantinuum Ltd. System Model H2 Product Data Sheet, 2024. URL: https://docs.quantinuum.com/systems/data_sheets/Quantinuum%20H2%20Product%20Data%20Sheet.pdf.

[19] IBM Quantum, 2021. URL: https://quantum.ibm.com/.

[20] J. M. Pino, J. M. Dreiling, C. Figgatt, J. P. Gaebler, S. A. Moses, M. S. Allman, C. H. Baldwin, M. Foss-Feig, D. Hayes, K. Mayer, C. Ryan-Anderson, and B. Neyenhuis. Demonstration of the trapped-ion quantum CCD computer architecture. *Nature*, 592(7853):209–213, April 2021. doi:10.1038/s41586-021-03318-4.

[21] David C. McKay, Christopher J. Wood, Sarah Sheldon, Jerry M. Chow, and Jay M. Gambetta. Efficient Z gates for quantum computing. *Phys. Rev. A*, 96(2):022330, August 2017. doi:10.1103/PhysRevA.96.022330.

[22] Quantinuum Ltd. System Operation, 2024. URL: https://docs.quantinuum.com/systems/user_guide/hardware_user_guide/system_operation.html.

[23] Unai Aseguinolaza, Nahual Sobrino, Gabriel Sobrino, Joaquim Jornet-Somoza, and Juan Borge. Error estimation in current noisy quantum computers. *Quantum Inf. Process.*, 23(5):181, May 2024. doi:10.1007/s11128-024-04384-z.

[24] Ying Li and Simon C. Benjamin. Efficient Variational Quantum Simulator Incorporating Active Error Minimization. *Phys. Rev. X*, 7(2):021050, June 2017. doi:10.1103/PhysRevX.7.021050.

[25] Ewout van den Berg, Zlatko K. Minev, Abhinav Kandala, and Kristan Temme. Probabilistic error cancellation with sparse Pauli–Lindblad models on noisy quantum processors. *Nat. Phys.*, 19(8):1116–1121, August 2023. doi:10.1038/s41567-023-02042-2.